

Docker Workshop

Lab 1

Working with images

1	COMMANDS COVERED	1
2	LAB SETUP	3
3	DOCKER IMAGE BASICS	4
3.1	CONTAINER REGISTRY.....	5
4	UNDERSTANDING IMAGES ON DOCKER HUB.....	5
4.1	REPOSITORIES, TAGS AND IMAGE REFERENCE NAME	6
4.2	UNDERSTANDING TAGGED IMAGE REFERENCES, MANIFEST LISTS AND DIGESTS	6
4.3	TAG TYPES AND ALIASES.....	9
4.4	REPOS FOR LANGUAGES AND LIBRARIES / FRAMEWORKS.....	11
4.5	REPOS FOR APPLICATIONS / DATABASES.....	13
5	DOWNLOADING (PULLING) IMAGES AND LISTING IMAGES	14
5.1	WORKING WITH WINDOWS IMAGES / CONTAINERS	16
5.2	WORKING WITH .NET DOCKER IMAGES.....	16
5.3	STORAGE OF DOWNLOADED IMAGES	17
6	IMAGE IDS AND FILTERING IMAGES	17
7	GETTING DETAILED INFORMATION ON IMAGES	18
8	DELETING IMAGES	19
9	PUSHING / PULLING IMAGES TO / FROM A DOCKER HUB ACCOUNT	20
10	DOCKER COMMAND CATEGORIES	22

1 Commands covered

Searching for images	Pulling images
<pre>docker search image-name docker search --filter "key=value" image-name</pre> https://www.tutorialworks.com/find-docker-images/	<pre>docker image pull image-name docker image pull -a repo-name</pre> https://www.educba.com/docker-pull/

https://www.configserverfirewall.com/docker/search-images/	

Listing and filtering images	Getting detailed info on images
<pre>docker image ls docker image ls REPO[:TAG] docker image ls --filter=reference=imagel docker image ls --filter "key=value" docker image ls --format go-template</pre> https://techtutorialsite.com/list-docker-images/	<pre>docker image inspect image-name/id docker image inspect -f go-template image-name/id</pre> https://adamtheautomator.com/docker-inspect/

Deleting images	Tagging images
<pre>docker image rm image-name / image-id / image-digest docker image rm -f image-name / image-id / image-digest docker image rm -f \$(docker image ls -q)</pre> https://www.digitalocean.com/community/tutorials/how-to-remove-docker-images-containers-and-volumes https://www.freecodecamp.org/news/how-to-remove-images-in-docker/	<pre>docker image tag source-image target-image</pre> https://blog.atomist.com/docker-image-tags/ https://www.freecodecamp.org/news/an-introduction-to-docker-tags-9b5395636c2a/

Logging into a remote registry	Pushing images
<pre>docker login</pre> https://www.techrepublic.com/article/how-to-successfully-log-in-to-dockerhub-from-the-command-line-interface/	<pre>docker push user-name/repo-name:tag</pre> https://www.section.io/engineering-education/docker-push-for-publishing-images-to-docker-hub/ https://jsta.github.io/r-docker-tutorial/04-Dockerhub.html

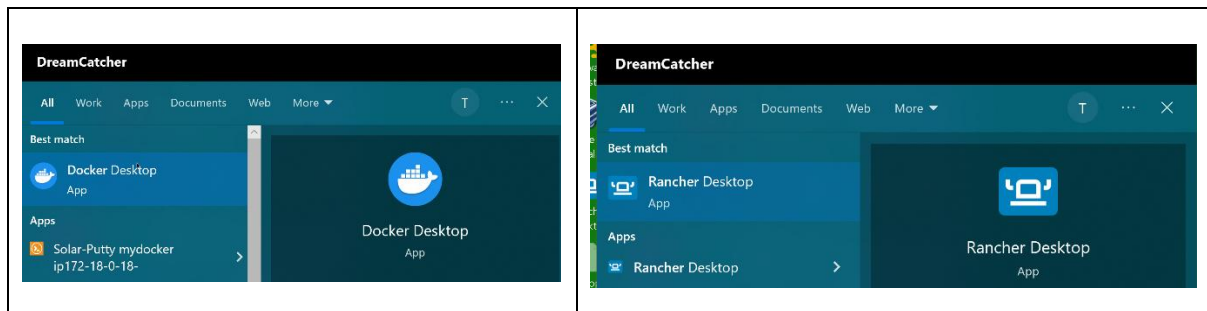
--	--

2 Lab setup

Make sure you already have an account on [DockerHub](#)

You should have an appropriate software installation to work with Docker containers on your machine: this could be either Docker Desktop, Rancher Desktop or some other suitable Docker runtime engine.

Start up any one (**but not BOTH !**) of these applications via Windows start menu.



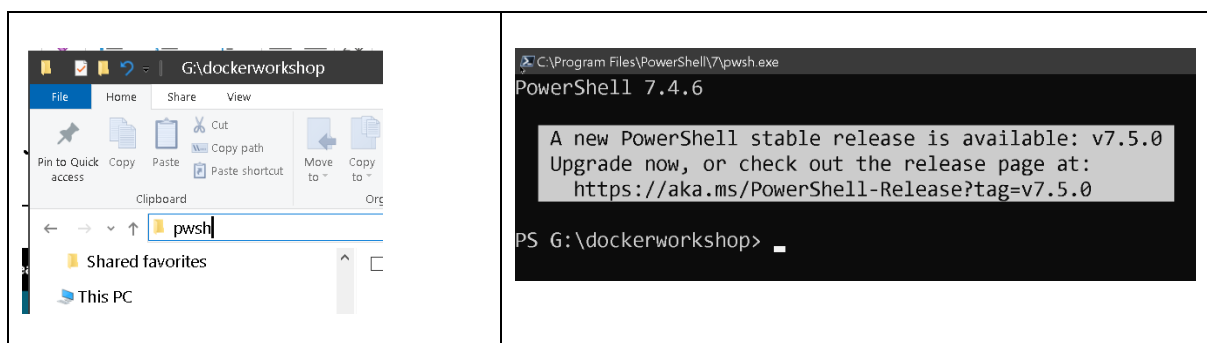
If you are using Rancher Desktop, ensure that you have removed all the K3s associated images in accordance with the installation instructions.

Create a suitable directory on your host machine to place the various files and source code for this workshop (for e.g. C:\dockerworkshop)

Open one or more PowerShell 7 terminal shell in that directory. Two approaches to do this:

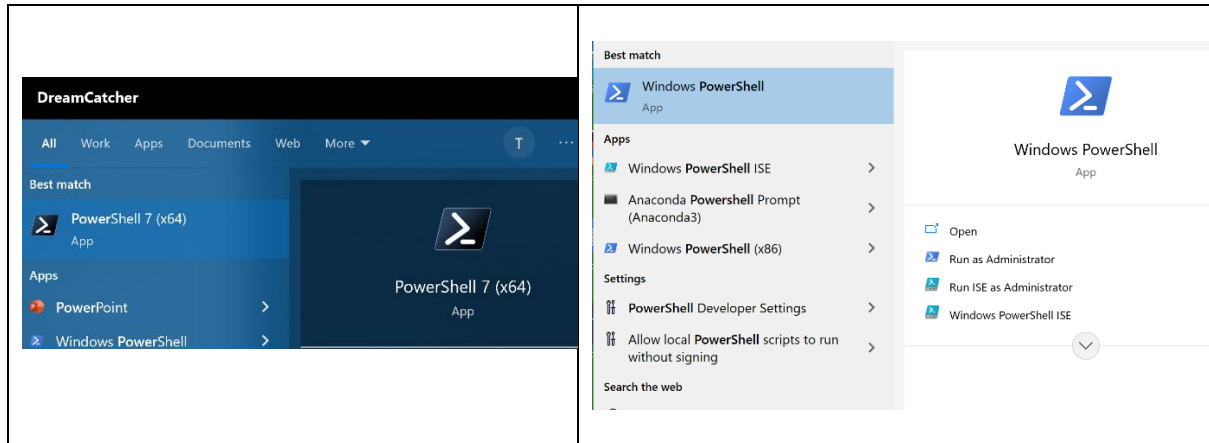
Approach 1

A quick shortcut to accomplish this is to open a File Explorer in that directory and type `pwsh` in the address bar.



Approach 2

Alternatively, you can start up the PowerShell 7 terminal (or Windows Powershell – the older version) from the Start menu of Windows.



Then use type the `cd` command to change to the workshop directory, for e.g

```
cd C:\dockerworkshop
```

Once you have a Powershell terminal open, the first thing we will do is to check the Docker version and to also quickly verify that the Docker client and daemon (engine) is running by typing:

```
docker version
```

Normal output would look similar to below:

```
Client:
 Cloud integration: v1.0.35+desktop.5
 Version:          24.0.6
 API version:      1.43
 Go version:       go1.20.7
 Git commit:       ed223bc
 Built:            Mon Sep  4 12:32:48 2023
 OS/Arch:          windows/amd64
 Context:          default

Server: Docker Desktop 4.25.0 (126437)
 Engine:
  Version:          24.0.6
  API version:      1.43 (minimum version 1.12)
  Go version:       go1.20.7
  Git commit:       1a79695
  Built:            Mon Sep  4 12:32:16 2023
  OS/Arch:          linux/amd64
  Experimental:     false
...
...
...
```

If the Docker daemon (engine) is not available, you will get an error message similar to the following:

```
error during connect: This error may indicate that the docker daemon is not running.: Get
"http://%2F%2F.%2Fpipe%2Fdocker_engine/v1.24/version": open //.pipe/docker_engine: The system
cannot find the file specified.
```

3 Docker image basics

A Docker image is a read-only, reusable template containing the files and configuration used to create and start a container. This container will provide the environment to host and execute an application or process of interest. Before running an application in a container, developers

must identify an appropriate image, choose a suitable version, download it, inspect its configuration and confirm that it is compatible with the intended environment.

Depending on the application or process that needs to be containerized, an image may contain:

- a) A minimal OS (Linux / Windows) user-space filesystem;
- b) Application binaries or source code;
- c) Language runtimes such as Java, Node.js or Python;
- d) Configuration defaults such as environment-variable definitions;
- e) An entry point or default command;
- f) Metadata such as exposed ports, working directory and labels.

Docker image	Docker container
Read-only, reusable template	Runtime instance created from an image
Normally stored locally or in a registry	Runs on a Docker Engine
Can be reused to create many containers	Has its own unique runtime state different from other running containers
Contains immutable filesystem layers	Adds a mutable, writable container layer
Identified by repository, tag, ID or digest	Identified by name or container ID
Does not consume CPU resources	Used to run one or more applications or processes of interest

3.1 Container registry

A registry stores and distributes container images. Registries may be:

- Public;
- Private;
- Self-hosted;
- Cloud-provider managed;
- Organization-specific.

[DockerHub](#) is the default public registry used in by the Docker CLI to download images from, if no explicit registry host is specified. Docker Hub hosts large repositories of pre-built images that can be pulled to a local (private) registry for immediate use.

Other examples of public registries include:

- GitHub Container Registry;
- Amazon Elastic Container Registry;
- Azure Container Registry;

4 Understanding images on Docker Hub

Go to [DockerHub](#), sign in to your Docker account and click on Explore to get a list of the images available (which will be sorted based on popular categories and trending).



Scroll down the list look for `Most pulled images` and select `View All`.

Most pulled images

[View all](#)

 postgres  The PostgreSQL object-relational... ☆10K+ ⬇1B+	 httpd  The Apache HTTP Server Project ☆4.8K ⬇1B+	 rabbitmq  RabbitMQ is an open source multi-protoco... ☆5.1K ⬇1B+	 nginx  Official build of Nginx. ☆10K+ ⬇1B+
--	--	---	--

You should be directed to a page with 2 filters applied: right now you are viewing only [official Docker images](#). These are a curated set of Docker repositories hosted on Docker Hub whose collection of images provide base starting point for users to create their own custom images.

4.1 Repositories, tags and image reference name

A repository is a collection of multiple related versions or variants of a particular image. These images are usually related to a base OS, popular programming language, popular development library /framework, popular database engine / web server, etc

```
alpine
ubuntu
python
nginx
mysql
node
```

These different image variants are provided for different requirements, such as:

- Different underlying base OS (Debian-based runtime, Alpine-based runtime, etc)
- Release types (stable, mainline, etc)
- The number of tools and runtime libraries (whether full or minimal)
- Development environment for applications (languages, etc)

A tag is a label associated with a particular image version or variant.

The repository and the tag are combined together in the form of `repo:tag` to provide a human readable tagged **image reference** that is used to uniquely identify an image variant.

4.2 Understanding tagged image references, manifest lists and digests

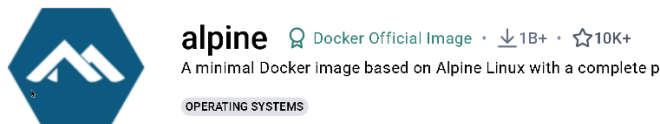
There are repositories for many popular Linux distro images here such as Ubuntu, Debian, Fedora and Alpine. The images in these repositories are commonly used as base images for building application containers or running short-lived administrative tasks.

Scroll down and look for Alpine and click on it.

Notice the format of the URL for the home page for this image. This indicates that it is a top-level or official repository:

https://hub.docker.com/_/alpine

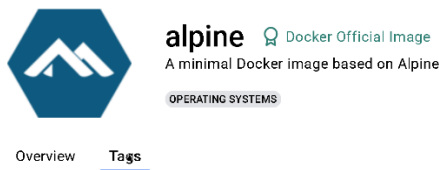
There is also an icon indicating the official Docker image status for this particular repo.



You can scroll through and get more information on this particular repo.

Alpine is a minimal Linux distro that contains many useful packages, utilities and tools that are frequently associated with Linux-related work, but with a much smaller resource footprint compared to larger Linux images such as Ubuntu. It is one of the most widely used based Linux distro for images containing other languages and library runtimes.

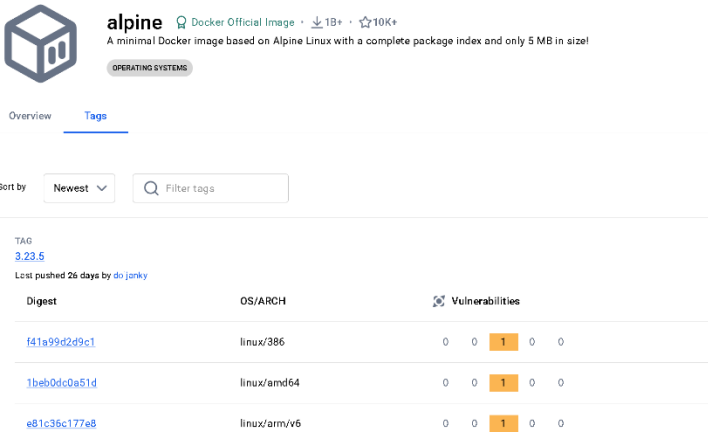
Click on the Tags option to see all the tags available for this particular repo.



In the upper right hand corner of each individual Tag box, you will see the Docker command to download images (`docker pull`) together with the **tagged image reference** (`repo:tag`) that uniquely identifies that particular image variant / version.

For e.g.: `docker pull alpine:3.23.5`

For a single given tagged image reference (for e.g. `alpine:3.23.5`), there are further different images of that variant for different types of OS/arch (platform). This is common for many images and is known as a **multiplatform image**.



alpine Docker Official Image · 1B+ · 10K+
A minimal Docker image based on Alpine Linux with a complete package index and only 5 MB in size!

Overview **Tags**

Sort by **Newest** Filter tags

TAG
3.23.5
Last pushed 26 days by [dojkeny](#)

Digest	OS/ARCH	Vulnerabilities
f41e99d2d9c1	linux/386	0 0 1 0 0
1beb0dc0a51d	linux/amd64	0 0 1 0 0
e81c36c177e8	linux/arm/v6	0 0 1 0 0

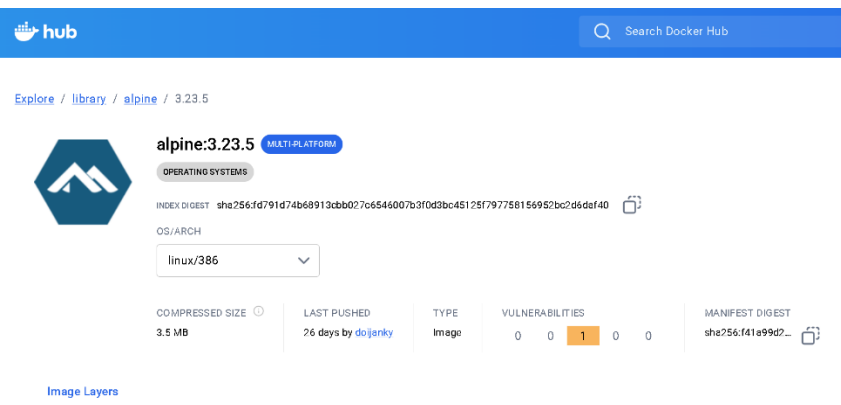
There is a platform-specific list called the **image manifest** which references the actual image content for a given platform (`linux/386`, `linux/amd64`, etc) on that list, which includes metadata such as:

- the image configuration
- the filesystem layers
- the layer sizes, etc

Every image manifest for a given platform is uniquely identified by a **manifest digest**, a SHA-256 hash sequence which is 64 characters long. This is what is shown in the listing above next to each platform.

The most common platform would be `linux/amd64`, which would refer to host machines using 64-bit CPU chips such as AMD Ryzen family processors, Intel Core i5, i7, Ultra and Xeon.

If you click on the specific tag link in the tags page, you will be transitioned to a layers page for that given tag, for e.g.



hub Search Docker Hub

Explore / library / **alpine** / 3.23.5

alpine:3.23.5 MULTI-PLATFORM
OPERATING SYSTEMS

INDEX DIGEST `sha256:f41e99d2d9c1b027c6546007b3f0d3bc45125f797758156952bc2d6daf40`

OS/ARCH
`linux/386`

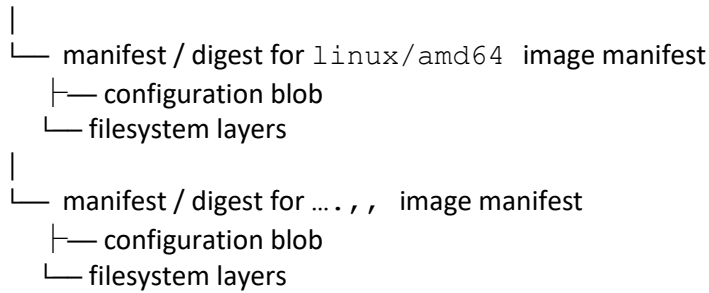
COMPRESSED SIZE	LAST PUSHED	TYPE	VULNERABILITIES	MANIFEST DIGEST
3.5 MB	26 days by dojkeny	Image	0 0 1 0 0	<code>sha256:f41e99d2...</code>

[Image Layers](#)

For every given tagged image reference (for e.g. `alpine:3.23.5`), there is a top-level **index digest** which is an identifier for the complete manifest list. This list is a **collection of the manifest digests and the image manifest** for each of the supported platforms mentioned earlier.

Tagged image reference (e.g. `alpine:3.23.5`)

- └─ top level index digest / manifest list
 - └─ manifest digest / `linux/386` image manifest
 - └─ configuration blob
 - └─ filesystem layers



Change between the platform (OS/Arch) options in the drop-down list. Notice that the index digest stays the same, while the manifest digest changes to reflect the same value that you see in the previous tags page.

Return to the main Overview page, and check the Tag Summary on the right hand side which gives you the top-level **index digest** for each of the image references in the drop down list.

Tag summary

Recent tags
3.24.1

Content type

Image

Digest

sha256:28bd5fe8b...

Size

3.7 MB

Last updated

about 1 month ago

docker pull alpine:3.24.1

4.3 Tag types and aliases

Return back to the main tag page.

As we saw earlier, the actual immutable identifier for the content for an image corresponding to a particular platform is the **manifest digest**:

TAG	
3.23.5	
Last pushed 26 days by dojanky	
Digest	OS/ARCH
f41a99d2d9c1	linux/386
1beb0dc0a51d	linux/amd64
e81c36c177e8	linux/arm/v6

When you download a particular image through the `docker pull` command, you will typically use the tagged image reference, rather than the actual digest, for e.g:

```
docker pull alpine:3.23.5
```

Docker automatically selects the appropriate platform-specific image corresponding to your host platform that your Docker engine is running on.

A tagged image reference however can be changed by the repository owner so that it points to a different list of manifest digests corresponding to different image contents at a later point of time. This depends on the tags:

Tag type	Expected behaviour	Example
Floating tag	Moves regularly	<code>alpine:latest</code>
Major-version tag	Moves to newer minor/patch releases	<code>alpine:3</code>
Minor-version tag	Moves to newer patch releases	<code>alpine:3.23</code>
Exact-version tag	Most stable, but may still be rebuilt	<code>alpine:3.23.5</code>
Digest	Never changes	<code>alpine@sha256:...</code>

Exact-version tags, which have a full version number in the form of `repo:x.y.z`, for e.g. `alpine:3.23.5` will usually not change.

The other tags will typically change over time and will usually reference another existing exact-version tag, and so are effectively **aliases for that exact tag**.

The `latest` tag is a floating tag found in some repos (but not all) which is used by the repo owner to identify the default image to be downloaded, if no tag is specified in the `docker pull` command when downloading an image.

For e.g. the command:

```
docker pull alpine
```

is interpreted by Docker as

```
docker pull alpine:latest
```

On the Alpine tags page: https://hub.docker.com/_/alpine/tags search for the `latest` tag.

Note down the digests for this tag for the various platforms

Then look around for other neighbouring tags that have identical digests

You are likely to find several tags that have identical digests with `latest`, for e.g.

- `3.24.1`
- `3.24`
- `3`
- `latest`

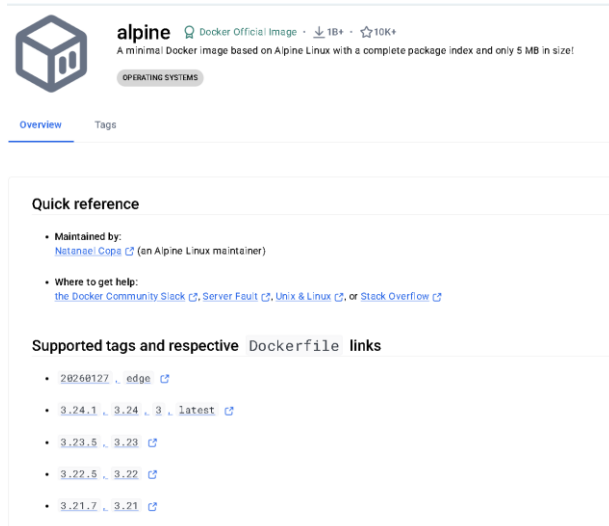
In this case, all these different tags reference the same list of digests for different platforms.

The tag with the complete version number is the exact tag: `3.24.1`

The others are all floating tags, which are effectively aliases for `3.24.1`

So, when you execute the `docker pull` command using any of these aliases in a later lab, you will download the **exact same image content**.

To see all the aliases for the same image, you can go back to the Overview page and check in the Supported tags and respective Dockerfile links section.



You can also verify in the Tag Summary on the right hand side which gives you the top-level **index digest** for each of the image references in the drop down list. Image references with the same top-level index digest value are effectively aliases for each other.

At a later point of time, the latest version for this repo could evolve to become 3.24.8. When that happens, it is likely that the repo owner will update the other floating tags 3.24, 3, latest to reference 3.24.8.

However, the exact-version tag 3.24.1 is likely to remain to reference the original image content.

Important points to keep in mind regarding the `latest` tag:

- Does not necessarily identify the newest or most stable version for that repo. Its meaning depends on the repo owner
- May not exist at all for certain repos

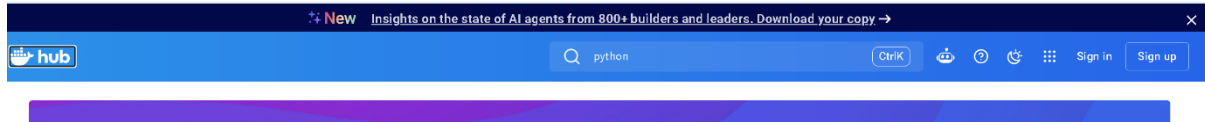
4.4 Repos for languages and libraries / frameworks

DockerHub provides Docker Official images for many popular programming languages and runtimes.

Language	Official repository
JavaScript / TypeScript	node
Python	python
Java	eclipse-temurin
PHP	php
C / C++	gcc

Search for the Python official Docker repo

You can use the search box at the middle to perform a search



In the Overview page, in the Simple Tags section, you will see a variety of tags with additional terms in their names such as `trixie`, `bookworm`, `rc`, `slim`, `alpine` and so on besides the version number.

The version number refers to the [actual Python version](#).

`trixie` and `bookworm` are Debian release codenames. They identify the specific Debian version on which the specific language of Python was installed on.

Tag component	Debian release	Current status
<code>trixie</code>	Debian 13	Current stable release
<code>bookworm</code>	Debian 12	Previous stable release, now receiving LTS support

There are also several other tags corresponding to older versions of Debian (11, 10, 9, etc) such as `bullseye`, `buster`, `stretch`, `jessie` and `wheezy` which are no longer actively supported.

The term `slim` means a reduced set of packages for that particular Debian release, for e.g. `3.12.13-slim-trixie` which results in a reduced image size

You can search for `trixie` / `bookworm` in the tags page (https://hub.docker.com/_/python/tags) and compare the image sizes for the tags with the `slim` term and without that term to verify this.

If you see the term `slim` alone without a corresponding Debian release codename, it usually means that latest Debian release is being used, which at this point is `trixie` (and could be another code name in the future)

`3.12.13-slim`

actually references

`3.12.13-slim-trixie`

The terms `b3` and `rc` indicate pre-release stages of Python. They describe the Python version inside the image, not the Debian or Docker image variant.

- `b3` means beta 3. A beta release is intended for testing. Major features are generally complete, but bugs and compatibility issues may remain.
- `rc` means release candidate. A release candidate is closer to the final release than a beta and is expected to become final unless significant problems are discovered.

For production work, avoid using `b3` and `rc` images, and opt for stable images.

For any given repo, you can get a specific explanation on the tag terminology through a prompt on an AI tool, for e.g.

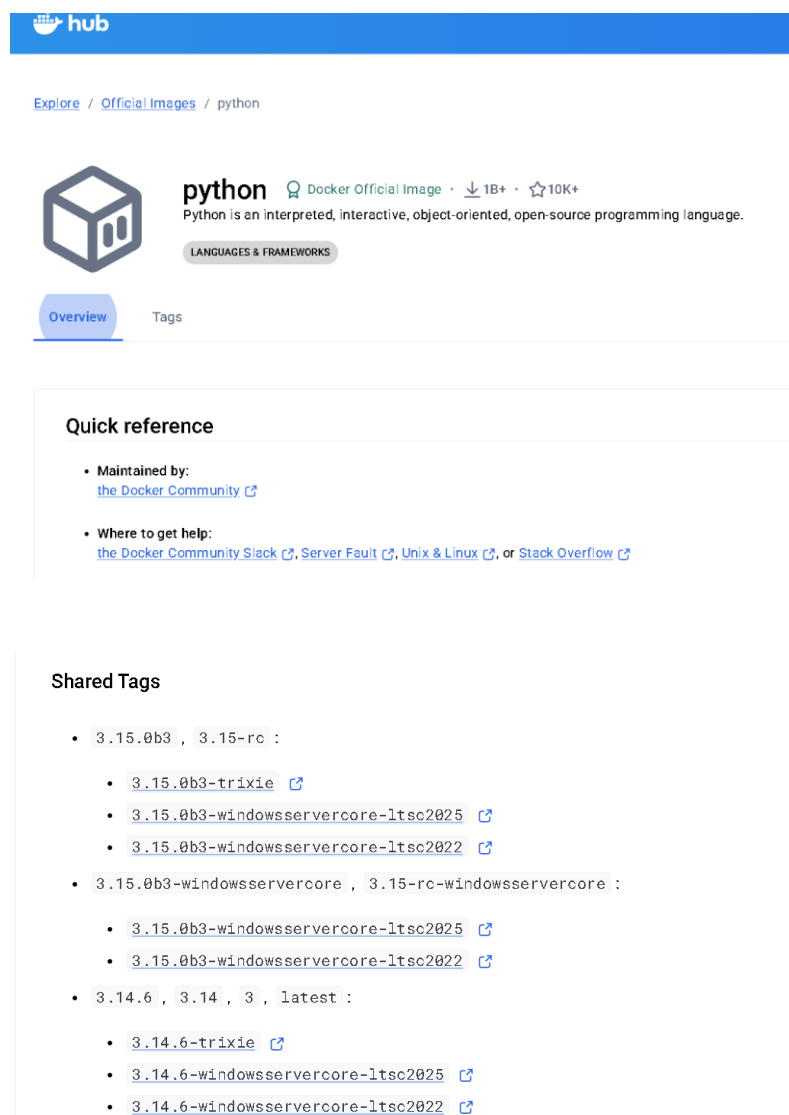
Explain in detail what this tag xxxx means in the Python DockerHub repository?

Python is also available installed on a Windows base, for which there are several possible [image options](#) possible.

For this particular repo, the option `windowservercore` is probably the most common.

More info on working with languages / applications installed on top of Windows base images will be provided in a later session.

If you want to check which exact tag the `latest` tag is aliased to, you can check for this in the Overview section of the main DockerHub page in the Shared Tags area.



hub

Explore / Official Images / python

 **python** Docker Official Image · 1B+ · 10K+
Python is an interpreted, interactive, object-oriented, open-source programming language.

LANGUAGES & FRAMEWORKS

Overview Tags

Quick reference

- Maintained by:
[the Docker Community](#)
- Where to get help:
[the Docker Community Slack](#), [Server Fault](#), [Unix & Linux](#), or [Stack Overflow](#)

Shared Tags

- 3.15.0b3 , 3.15-rc :
 - [3.15.0b3-trixie](#)
 - [3.15.0b3-windowsservercore-ltsc2025](#)
 - [3.15.0b3-windowsservercore-ltsc2022](#)
- 3.15.0b3-windowsservercore , 3.15-rc-windowsservercore :
 - [3.15.0b3-windowsservercore-ltsc2025](#)
 - [3.15.0b3-windowsservercore-ltsc2022](#)
- 3.14.6 , 3.14 , 3 , latest :
 - [3.14.6-trixie](#)
 - [3.14.6-windowsservercore-ltsc2025](#)
 - [3.14.6-windowsservercore-ltsc2022](#)

4.5 Repos for applications / databases

Repos for popular databases

Product	Type	Docker Hub repository
PostgreSQL	Relational SQL database	postgres
MySQL	Relational SQL database	mysql
MariaDB	MySQL-compatible SQL database	mariadb
MongoDB	Document-oriented NoSQL database	mongo
Redis	In-memory data store and cache	redis
Memcached	Distributed memory cache	memcached

Repos for popular web servers

Application	Purpose	Repository
NGINX	Web server, reverse proxy and load balancer	nginx
Apache HTTP Server	General-purpose web server	httpd
Traefik	Container-aware reverse proxy and edge router	traefik
HAProxy	TCP/HTTP load balancer and reverse proxy	haproxy

The explanation for the terminology of the tags in all these repos are conceptually very similar to the case of Python. You will have typically have:

- A version number for the particular database / application / webserver
- A code name for the base OS (Linux / Windows)
- Related terms for release versions, or slimmed down version of the base OS

Take some time to explore any of these repos and validate this for yourself.

5 Downloading (pulling) images and listing images

We download (or pull) images from the default public registry (Docker Hub) by specifying the tagged image reference (combination of `repo:tag`). Lets pull down a few light weight Linux distro images.

Search the official Alpine repository and identify a suitable image to download:

https://hub.docker.com/_/alpine/tags

For e.g. one with a suitable recent tag: 3.21

Type:

```
docker pull alpine:3.21
```

In this process, Docker will

- Retrieve the manifest list for the `alpine:3.21` tag
- Detect the host platform that the Docker engine is running on (for e.g. `linux/amd64`)
- Finds the matching platform entry from this manifest list
- Retrieves the platform-specific manifest
- Downloads the configuration and layers referenced by that manifest.

You should see an output similar to the following confirming a successful download of the image:

```
3.19: Pulling from library/alpine
46b060cc2620: Pull complete
Digest: sha256:95c16745f100f44cf9a0939fd3f357905f845f8b6fa7d0cde0e88c9764060185
Status: Downloaded newer image for alpine:3.19
```

```
docker.io/library/alpine:3.19
```

These downloaded image will be stored in Docker's local image store.

To get a list of all images available on the local image store (currently only the single image), type:

```
docker images
```

You should see a listing similar to the following:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
alpine	3.23	494edff73605	10 days ago	7.4MB

If you only specify the repo portion without a tag part for your image name, then Docker will automatically use the term `latest` for the tag, as explained earlier. If no such tag exists on the repo, the pull attempt will fail.

Try typing just:

```
docker pull alpine
```

The output messages should confirm it is pulling down the image with the `latest` tag

```
Using default tag: latest
latest: Pulling from library/alpine
Digest: sha256:0a4eaa0eecf5f8c050e5bba433f58c052be7587ee8af3e8b3910ef9ab5f9f5
Status: Downloaded newer image for alpine:latest
docker.io/library/alpine:latest
```

Check again that you now have two images with:

```
docker images
```

You should be able to view these images as well in the UI of your specific Docker installation (Docker Desktop / Rancher Desktop)

Go to the `alpine` and `busybox` repo and pull down a few more random images by searching through their respective tag pages.

Try to pull down images with tags which are aliases for exact tags, so that you are essentially downloading the same image content (see previous lab explanation on this). For e.g. try to pull down images with tags that have the same digest with the `latest` tag and see what happens.

In the process of downloading these image layers for these various images, Docker will be able to identify whether an image layer for an image to be downloaded already exists on the local store from a previous download image. If so, it will not need to download that particular layer. This is because images can share identical layers in their construction

For pulling images, you either use a CLI command as demonstrated previously or use the UI of your specific Docker installation (Docker Desktop / Rancher Desktop)

Try mixing both approaches to get a feel for them.

5.1 Working with Windows images / containers

The vast majority of images in the official Docker repos for applications / databases / etc utilize a base Linux OS (popular examples are Alpine, Ubuntu and Debian – with the codenames `bookworm`, `trixie` etc as we discussed earlier).

However, it is also possible to download Windows OS images as well as images of applications that run [on top of a base Windows OS image](#).

For e.g. Windows NanoServer images are available for [download from DockerHub](#).

Windows Server Core containers require a Windows host because they share a Windows kernel. Microsoft also imposes compatibility requirements between the Windows host version and the container image version.

The Docker installation you are using will determine whether you can work with Windows images / containers.

- Docker Desktop is capable of working with **both Linux and Windows** images / containers
- Rancher Desktop is capable of working **with only Linux images** / containers

5.2 Working with .NET Docker images

Microsoft provides [official .NET Docker images](#) which are publicly available on the [Microsoft Artifact Registry](#). These are useful for deploying a modern cross platform .NET C# application.

These images begin with: `mcr.microsoft.com/dotnet/`

More info on the [.NET container images are provided here](#).

There is also an official [DockerHub catalog](#) which provides links to the relevant tags for the different .NET families in the dotnet-docker GitHub repo.

You can also find the same related tag info at the [related Docker Hub repositories](#) for the different .NET families.

These repos merely list the tags; the actual images are pulled from the [Microsoft Artifact Registry](#).

Docker Hub repository	Image used in Dockerfiles	Main purpose
<code>microsoft/dotnet</code>		Overview or landing repository for the official .NET image family
<code>microsoft/dotnet-sdk</code>	<code>mcr.microsoft.com/dotnet/sdk</code>	Developing, restoring, compiling, testing and publishing .NET applications
<code>microsoft/dotnet-aspnet</code>	<code>mcr.microsoft.com/dotnet/aspnet</code>	Running ASP.NET Core web applications, APIs and services

microsoft/dotnet-runtime	mcr.microsoft.com/dotnet/runtime	Running framework-dependent console applications and worker services
microsoft/dotnet-runtime-deps	mcr.microsoft.com/dotnet/runtime-deps	Running self-contained or Native AOT applications; contains native OS dependencies but not the .NET runtime
microsoft/dotnet-monitor	mcr.microsoft.com/dotnet/monitor	Collecting diagnostic data such as traces, dumps, metrics and logs from running .NET applications
microsoft/dotnet-samples	mcr.microsoft.com/dotnet/samples	Prebuilt sample .NET and ASP.NET Core applications for demonstrations and testing

The default .NET SDK images [use Ubuntu if no OS is specified](#).

For e.g. to get the run time image with a base Ubuntu OS, you could run:

```
docker pull mcr.microsoft.com/dotnet/runtime:10.0
```

Note:

- .NET Framework (Versions 1.0 – 4.8): The legacy, original version. It is Windows-only.
- .NET Core (Versions 1.0 – 3.1): The total rewrite built from scratch to be open-source and cross-platform (Linux / Windows)
- .NET (Versions 5.0 – 10.0): the main, unified platform which is the direct evolution of .NET Core.

5.3 Storage of downloaded images

Downloaded images from DockerHub public registry are stored in a local store at specific locations depending on the OS of the host system (Windows / Linux / MacOS) as well as the particular Docker installation used.

For the case of Docker Desktop, [these are the locations of the images](#) for Linux and Windows.

For the case of Rancher Desktop, [these are the locations of the images](#) for Linux and Windows.

6 Image IDs and filtering images

We can check again on all images available on the local store after performing all the previous pulls:

```
docker images
```

To view only images from a specific repo, you could type:

```
docker images alpine
```

```
docker images busybox
```

The image ID is the actual unique identifier for a distinct image. A single unique image can have multiple different image references associated with it, which are in the `repo:tag` form. This should be the case if you had completed the previous lab properly where you pulled down images with tags that are aliases for exact tags, so that you are essentially downloading the same image content

You can add the `--filter` option to filter a long list of images, for e.g.

To view images with the latest tag:

```
docker images --filter=reference="*:latest"
```

To view all alpine images that are minor subversions within version 3.23:

```
docker images --filter=reference="alpine:3.23*"
```

To see all the options available for the `docker images` command (or any docker command in general), just type:

```
docker images --help
```

You should also be able to view and filter images using the UI of your specific Docker installation (Docker Desktop / Rancher Desktop)

Images are also viewable via the VS Code Docker extension UI, but no filtering functionality above.

7 Getting detailed information on images

We can get detailed information on an image with:

```
docker inspect repo:tag
```

You can alternatively also use the first 3 - 4 letters of the image ID to identify the image as well in this command (and any other commands that require you to identify a specific image), for e.g.

```
docker inspect imageid
```

Of particular interest is the `Cmd` section, which shows the commands that will be executed when a container is created and run from this particular image. We will be examining this in more detail in an upcoming lab

```
...  
...
```

```
    "Cmd": [
```

```

        "/bin/sh",
        "-c",
        "# (nop) ",
        "CMD [\"/bin/sh\"]"
    ],
    ...
    ...

```

You can also drill down into the output using a Go template expression that is passed to the `--format` option, as an example:

```
docker inspect --format '{{.Config.Cmd}}' alpine:3.21
```

```
docker inspect --format '{{.Config.Env}}' imageid
```

Images are also be inspected via the VS Code Docker extension UI

8 Deleting images

Deleting an image will remove the image and all of its layers from the Docker host. However, if an image layer is shared by more than one image, that layer will not be deleted until all images that reference it have been deleted.

If the image you are trying to delete is in use by a running container you will not be able to delete it. You will have to stop and delete any containers before trying the delete operation again. We will study running containers in the next lab.

Identify an image reference with a unique image ID (there is no other different image reference that has the same image ID), and delete it with:

```
docker rmi repo:tag
```

The output messages will indicate that the delete operation starts first in an untagging of the existing tag for this image followed by the deletion of all corresponding image layers that comprise this image.

```

Untagged: alpine:3.17.9
Untagged: alpine@sha256:ef813b2faa3dd1a37f9ef6ca98347b72cd0f55e4ab29fb90946f1b853bf032d9
Deleted: sha256:91dce2778e2dd8d10b7f0788e874976006127847e156f7f40d38694225f43321
Deleted: sha256:76367d75676f0ab56722a770f40a80941396ff850244f7659bb2b2fe06b125aa

```

If you attempt to delete an image that is referenced to by multiple image references by one of its existing tags, that tag is simply untagged but the image still remains. Identify an image such as this and attempt to delete it with one of its tags:

```
docker rmi repo:tag
```

The only message that comes out is:

```
Untagged: xxxx:yyyyy
```

If we check again with

```
docker images
```

You will see the other tags that are associated with that same image ID still remain:

If you now proceed to delete all the other tags associated with that single unique image ID, with successive `docker rmi` commands:

You will see that the only upon the deletion of the final tag is there an actual deletion of the corresponding image layers for that associated image ID (as shown in the output messages)

To remove ALL unused images (i.e. that are not being used or referenced by a running container) we can execute:

```
docker image prune -a
```

Respond `y` to the prompt that appears

Check that there no more images remaining in the local repository after this with:

```
docker images
```

9 Pushing / pulling images to / from a Docker Hub account

In order to upload (or push) an image from your local registry to Docker Hub (or some other configured remote registry), you will need to give a reference in the form of:

```
<dockerhubusername>/<repo-name>[:tag]
```

Pull down a light-weight image from any repo of your choice, for e.g.

```
docker pull busybox:latest
```

We can create a new reference (in the form of `repo:tag`) to this downloaded image with:

```
docker tag busybox:latest dockerhubusername/coolimage:v1
```

Check that the new tag has been created correctly with:

```
docker images
```

Make sure you are **logged into your DockerHub account** in an open active browser tab.

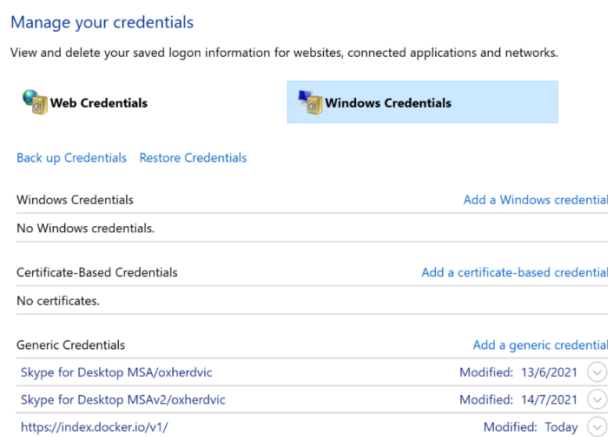
Next, perform a manual login at the CLI with:

```
docker login
```

and enter your username and password combination.

Once you have performed a manual login at the CLI successfully, Docker will automatically use the system credential manager to cache your username and password so that you do not need to repeat entering them for future push attempts.

On Windows, you can view the login credentials at: Control Panel -> User Accounts -> Manage Windows Credentials. In the area Generic Credentials, you should see some credentials related to Docker (<https://index.docker.io/v1/>). You will need to remove these credentials if you wish to push to a different Docker Hub account in the future from the same user account.



Next push the image by typing:

```
docker push dockerhubusername/coolimage:v1
```

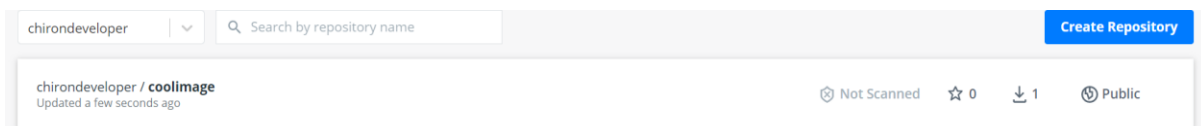
You should be able to see some messages indicating that the push was successful.

If you encounter any errors related to authentication when you attempt the push above, check that you are correctly logged into DockerHub account in your browser. If the error still persists,

Notice in the messages that appear that Docker is able to figure out that this image content is identical to an existing image in the DockerHub registry (`busybox:latest`) based on its SHA digest, and so it uses that available image instead of actual uploading the current image on the local store.

This prevents unnecessary uploads and significantly saves bandwidth. Of course, if you were uploading a custom image that does not exist in the DockerHub registry (which we will be doing at a later point in this workshop), then the actual image will need to be uploaded.

You should now be able to see this pushed image in the `coolimage` repo on your Docker Hub account.



Lets go ahead and delete the single image on our local registry which is referenced by 2 different tags at the moment with this command (or use the UI of your Docker installation)

```
docker rmi -f imageid
```

Check that it is deleted

```
docker images
```

And then finally pull down the image that you just uploaded to your Docker Hub account:

```
docker pull dockerhubusername/coolimage:v1
```

The VS Code Docker extension UI allows you to connect to a valid DockerHub account, view remote repos on that account, pull remote images from there into your local registry and push appropriate tagged images from your local registry to there.

You can try repeating the previous exercise using the VS Code Docker Extension UI if you wish.

10 Docker command categories

Type:

```
docker
```

You will see a long list which can be divided into 3 categories: common commands, management commands and commands

Common Commands:

run	Create and run a new container from an image
exec	Execute a command in a running container
ps	List containers
...	

Management Commands:

builder	Manage builds
buildx*	Docker Buildx (Docker Inc., v0.14.1)
...	
...	

Commands:

attach	Attach local standard input, output, and error streams to a running container
--------	---

The common commands, as the name suggest, are commands that you will very commonly use in working with containers and were present since Docker was introduced. The Docker management commands represent a later logical reorganization of the original common commands from earlier versions of Docker, are used with an additional command action, for which you can get a list of by typing the first management command.

For e.g. type

```
docker container
```

You see a whole list of subcommands related to managing containers. Try getting help on one of these subcommands, for e.g:

```
docker container ls --help
```

The equivalent older common command for this is `docker ps` (you will see this listed as well as an alias for `docker container ls`). Try getting help on that by typing:

```
docker ps --help
```

You will also see `docker container ls` (and a variety of other commands) listed as aliases for this as well. Notice that the explanation and options for both these commands (`docker container ls` and `docker ps`) are identical, i.e. both of them are functionally equivalent.

In the labs for this workshop, we will be using commands from all these different categories. In particular, we will be using the common commands (`docker run`, `docker ps`, `docker images`, etc) as opposed to their newer versions (`docker container run`, `docker container ls`, `docker image ls`) since the more common commands are more popular and widely used in documentation and tutorials on the web and Youtube as opposed to their newer versions.

However, if you are in any doubt about the which commands are considered to be equivalent, just run the commands with `--help` as previously demonstrated.

There is also a short list of equivalencies [available here](#):